# Chapter 16 exercises • Virtual machines

## OS16-E01 • Modes and virtual interrupts

Use an x86 teaching model: the guest runs in protected mode with EFLAGS.VM=0 and CR4.PVI=0. Its non-root kernel has CPL=0; its application has CPL=3; both have IOPL=0. The VMM runs in root context and intercepts guest CLI. It emulates a legal kernel CLI by updating this guest's virtual interrupt-enable state and injects the guest exception required for an illegal operation.

1. Should host interrupts remain disabled forever? What happens when the guest application executes CLI?
2. What gap remains if a sensitive instruction does not trap and differs from native guest semantics?

## OS16-E02 • Addresses and permission intersection

Pages4096 B, GVA0x2ABC, guestPTE[2]=(GFN7, rw), EPT[7]=(MFN0x12, r).

1. Find GPA, HPA, and read/write decisions.
2. Guest changes PTE[2] to GFN8, which does not map to MFN 0x12, but shadow remains VPN2→MFN 0x12. What fails, and which cached translations need attention?

## OS16-E03 • General nested-walk cost

Guest has 3 levels and EPT has 4. All page tables and the data page are resident, mappings and permissions are valid, and no faults occur. All TLB/walk caches miss; no large pages. Count PTE reads and the final data read only, excluding possible accessed/dirty updates.

1. Count table references and references including data.
2. Why not3+4=7?

## OS16-E04 • Four-slot PV ring

Use ordered single-producer/consumer model, N=4, req_prod7, req_cons6, rsp_prod5, rsp_cons4.

1. Find queued, outstanding, response, used, free, and next request slot.
2. After another submission, can one more submit? What changes after consuming one response?
3. Specify descriptor publication, slot reuse, and safe sleep while waiting for requests.

## OS16-E05 • Finite two-layer scheduling trace

One pCPU; A and B each have one vCPU and a continuously runnable job needing 20 ms CPU. Context switching costs zero. A runs 0–10 ms, B 10–20 ms, alternating by 10 ms; this finite trace stops after both jobs finish.

1. When does each finish? Does average share determine finite completion time?
2. Can high guest-thread priority unconditionally remove these waits?

## OS16-E06 • Balloon conservation

Host 8192 MiB, VMM reserves 1024 MiB, and A/B each demand 4096 MiB resident backing, not yet fully allocated. Nominal sizes stay fixed. The balloon driver reserves the reclaimed guest pages, and the VMM prevents access to another guest’s frame; no specific response to accidental access is assumed.

1. How much reduction is required? Account for 512 ballooned in each guest.
2. Can workloads still use every ballooned page without cost?

## OS16-E07 • Sharing and COW

Initially A:g3→h9 and B:g4→h11 contain identical bytes X. CPUs or DMA may write concurrently: stop these writers and invalidate stale writable CPU and second-stage translations before comparison and remapping.

1. Is a hash match enough to free h11? Give B's first-write ordering.
2. What if B retains a pre-merge writable TLB entry pointing to h11? Contrast it with A retaining a writable entry pointing to h9.

## OS16-E08 • Migration convergence and pause

Memory M=512 MiB and copy bandwidth is 128 MiB/s. In each round, exactly 32 MiB/s × that round’s duration of distinct pages become dirty and are tracked for retransmission. Perform three rounds, freeze all writers, then copy the residual. Ignore control and CPU/device-state costs; shared storage is available at the target.

1. List rounds, residual, total bytes, and pause.
2. Does dirtying128 MiB/s shrink the model?
3. What if target starts, acknowledgment is lost, and source resumes?

## OS16-E09 • Snapshots and external effects

Base={0:A,1:B}, delta={1:C}. Sent mail is outside VM state.

1. Evaluate active reads and rollback to base. What happens to mail?
2. Is copying an active disk file necessarily an application-consistent backup?

## OS16-E10 • Abstractions and bytecode

Requirements are testing another kernel, running an old cross-ISA binary, running a Java class, and isolating two same-kernel web services. The bytecode belongs to an otherwise valid complete method with at least two local-variable slots.

1. Choose mechanisms and identify shared layers.
2. Trace iconst_2, iconst_3, iadd, istore_1. Explain iadd on an empty stack.

### References and attribution

Lin Cing, handwritten operating systems notes. Silberschatz et al., Operating System Concepts, 9th ed., Ch. 16. Xen SOSP 2003. ESX memory management OSDI 2002. JVM SE7 verifier.

Original educational synthesis by Lin Cing, licensed under CC BY 4.0.
https://github.com/xinhi6716/computer-science-self-study